

Docker challenge

1. Create a customized Docker image using a Dockerfile.

Create a Dockerfile:

```
ubuntu@ip-172-31-10-98:~$ vi Dockerfile
ubuntu@ip-172-31-10-98:~$ cat Dockerfile
FROM nginx:alpine

WORKDIR /usr/share/nginx/html

COPY . .

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Build : docker build -t frontend:latest

```
ubuntu@ip-172-31-10-98:~$ docker build -t frontend-app:latest .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon   94.9MB
Step 1/5 : FROM nginx:alpine
alpine: Pulling from library/nginx
fbaed3f7fcbe: Pulling fs layer
de1b677d8c00: Pulling fs layer
94d083cf706a: Pulling fs layer
e654dbbbb9e1: Pulling fs layer
a5008f4a4b25: Pulling fs layer
43f834d60d8a: Pulling fs layer
abaae85d1626: Pulling fs layer
e654dbbbb9e1: Download complete
a5008f4a4b25: Download complete
43f834d60d8a: Download complete
fbaed3f7fcbe: Download complete
de1b677d8c00: Download complete
94d083cf706a: Download complete
abaae85d1626: Download complete
```

```
Step 2/5 : WORKDIR /usr/share/nginx/html
----> Running in d270ab4d7c1c
----> Removed intermediate container d270ab4d7c1c
----> b9cda2645323
Step 3/5 : COPY . .
----> 4bd54d283b4d
Step 4/5 : EXPOSE 80
----> Running in a7e51f57882f
----> Removed intermediate container a7e51f57882f
----> 5b22effe3d20
Step 5/5 : CMD ["nginx", "-g", "daemon off;"]
----> Running in de22408a2efd
----> Removed intermediate container de22408a2efd
----> 675db38b5e36
Successfully built 675db38b5e36
Successfully tagged frontend-app:latest
```

Run : `docker run -d -p 83:80 frontend:latest`

```
ubuntu@ip-172-31-10-98:~$ docker run -d -p 83:80 frontend-app:latest
6d49c107e2b01ebe8c7e815411a7e6c491496ee85e8c363e902a20cd54fafd97
ubuntu@ip-172-31-10-98:~$
```

2. Push the image to Docker Hub.

Tag : `docker tag frontend-app:latest`
`bhargav0117/frontend-app:latest`

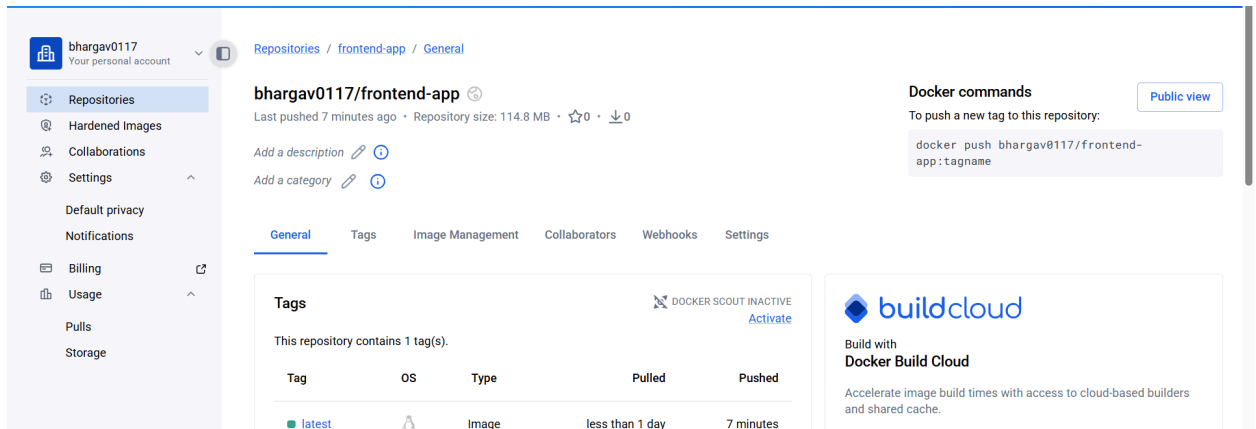
```
ubuntu@ip-172-31-10-98:~$ docker tag frontend-app:latest bhargav0117/frontend-app:latest
ubuntu@ip-172-31-10-98:~$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
bhargav0117/custom-nginx:v1	cb423984a499	292MB	85.1MB	
bhargav0117/frontend-app:latest	675db38b5e36	282MB	120MB	U
custom-nginx:v1	a84660173250	342MB	94.6MB	
eclipse-temurin:17-jre-alpine	b0ae54a36f82	257MB	68.9MB	
frontend-app:latest	675db38b5e36	282MB	120MB	U
jenkins/jenkins:lts	01c992ffef29	799MB	289MB	U
maven:3.9.6-eclipse-temurin-17	29a1658b1f30	728MB	221MB	
nginx:alpine	8b1e78743a03	93.6MB	27MB	
nginx:latest	5aca99593157	241MB	66MB	
springboot-multi:v1	a4f3cfb0c0f9	462MB	126MB	
springboot-single:v1	b578b94a53fa	871MB	283MB	
test-nginx:latest	92c0e1ba605b	238MB	63.1MB	
yourdockerhub/frontend-app:latest	675db38b5e36	282MB	120MB	U

Push : `docker push bhargav0117/frontend-app:latest`

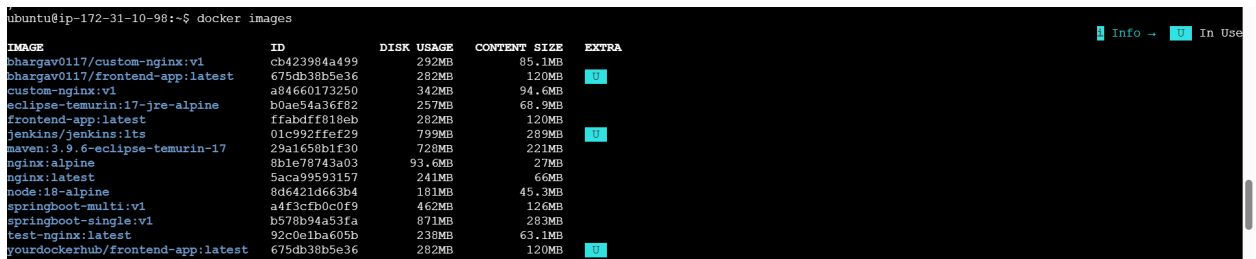
```
ubuntu@ip-172-31-10-98:~$ docker push bhargav0117/frontend-app:latest
The push refers to repository [docker.io/bhargav0117/frontend-app]
fbaed3f7fcbe: Layer already exists
delb677d8c00: Layer already exists
6a0ac1617861: Layer already exists
94d083cf706a: Layer already exists
43f834d60d8a: Layer already exists
18682be42827: Layer already exists
e654dbbbb9e1: Layer already exists
a5008f4a4b25: Layer already exists
abaae85d1626: Layer already exists
latest: digest: sha256:675db38b5e36960c3fc3a2ccb742c954179ea6f8a84ec717ee50e95f50d1a180 size: 2266
ubuntu@ip-172-31-10-98:~$
```

Verify:



3. Push the same image to Amazon ECR.

Docker images:

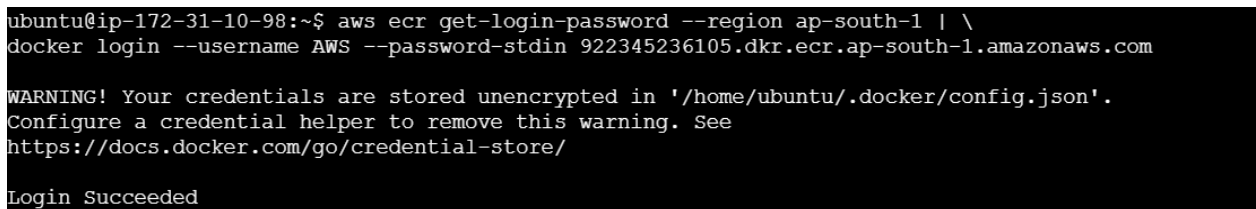


Login to AWS ECR :

aws ecr get-login-password --region ap-south-1 | \

docker login --username AWS --password-stdin

<account-id>.dkr.ecr.ap-south-1.amazonaws.com



Create Repository : aws ecr create-repository --repository-name frontend-app

```
ubuntu@ip-172-31-10-98:~$ aws ecr create-repository --repository-name frontend-app
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:ap-south-1:922345236105:repository/frontend-app",
    "registryId": "922345236105",
    "repositoryName": "frontend-app",
    "repositoryUri": "922345236105.dkr.ecr.ap-south-1.amazonaws.com/frontend-app",
    "createdAt": "2026-06-04T13:33:06.208000+00:00",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": false
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}
```

Tag Image : docker tag node-app:v1

<account-id>.[dkr.ecr.ap-south-1.amazonaws.com](https://922345236105.dkr.ecr.ap-south-1.amazonaws.com/)/frontend:latest

Push Image : docker push

<account-id>.[dkr.ecr.ap-south-1.amazonaws.com](https://922345236105.dkr.ecr.ap-south-1.amazonaws.com/)/frontend:latest

```
ubuntu@ip-172-31-10-98:~$ docker tag frontend-app:latest 922345236105.dkr.ecr.ap-south-1.amazonaws.com/frontend-app:latest
ubuntu@ip-172-31-10-98:~$ docker push 922345236105.dkr.ecr.ap-south-1.amazonaws.com/frontend-app:latest
The push refers to repository [922345236105.dkr.ecr.ap-south-1.amazonaws.com/frontend-app]
3f1c9766ec09: Pushed
43f834d60d8a: Pushed
fbaed3f7fcbe: Pushed
94d083cf706a: Pushed
a5008f4a4b25: Pushed
abaae85d1626: Pushed
de1b677d8c00: Pushed
6a0ac1617861: Pushed
e654dbbb9e1: Pushed
latest: digest: sha256:ffabdf818eb801ae32ea9750dbff6bc73492ab40a825b1cb416a7778fe4749b size: 2266
ubuntu@ip-172-31-10-98:~$
```

4. Provision one EC2 instance using Terraform and install Jenkins.

Update system : sudo apt update && sudo apt install -y gnupg software-properties-common curl

```

ubuntu@ip-172-31-10-98:~$ sudo apt install -y gnupg software-properties-common curl
gnupg is already the newest version (2.4.8-4ubuntu3).
software-properties-common is already the newest version (0.120).
software-properties-common set to manually installed.
curl is already the newest version (8.18.0-1ubuntu2.1).
curl set to manually installed.
Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 3
ubuntu@ip-172-31-10-98:~$ curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
ubuntu@ip-172-31-10-98:~$ echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list
deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com resolute main
ubuntu@ip-172-31-10-98:~$ sudo apt update
sudo apt install terraform -y
Hit:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease

```

Add HashiCorp GPG key : curl -fsSL

https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg

Add Terraform repository : echo "deb

[signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]

https://apt.releases.hashicorp.com \$(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list

```

ubuntu@ip-172-31-10-98:~$ curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
ubuntu@ip-172-31-10-98:~$ echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list
deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com resolute main

```

Install Terraform : sudo apt update

sudo apt install terraform -y

```

ubuntu@ip-172-31-10-98:~$ sudo apt update
sudo apt install terraform -y
Hit:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:3 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease
Get:4 https://aquasecurity.github.io/trivy-repo/deb resolute InRelease [3238 B]
Get:5 https://apt.releases.hashicorp.com resolute InRelease [11.2 kB]
Err:4 https://aquasecurity.github.io/trivy-repo/deb resolute InRelease
The following signatures couldn't be verified because the public key is not available: NO_PUBKEY 35B8ACA44FD9CA9F
Get:6 https://apt.releases.hashicorp.com resolute/main amd64 Packages [237 kB]
Hit:7 http://security.ubuntu.com/ubuntu resolute-security InRelease
Warning: OpenPGP signature verification failed: https://aquasecurity.github.io/trivy-repo/deb resolute InRelease: The following signatures couldn't be verified because the public key is not available: NO_PUBKEY 35B8ACA44FD9CA9F
Error: The repository 'https://aquasecurity.github.io/trivy-repo/deb resolute InRelease' is not signed.
Notice: Updating from such a repository can't be done securely, and is therefore disabled by default.
Notice: See apt-secure(8) manpage for repository creation and user configuration details.
Notice: Some sources can be modernized. Run 'apt modernize-sources' to do so.
Installing:
  terraform
Summary:
Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 3
Download size: 34.8 MB
Space needed: 117 MB / 41.1 GB available

```

Verify installation : terraform -version

```

ubuntu@ip-172-31-10-98:~$ terraform -version
Terraform v1.15.5
on linux amd64
ubuntu@ip-172-31-10-98:~$ aws --version
aws-cli/2.31.35 Python/3.14.4 Linux/7.0.0-1006-aws source/x86_64.ubuntu.26
ubuntu@ip-172-31-10-98:~$ mkdir terraform
ubuntu@ip-172-31-10-98:~$ cd terraform/
ubuntu@ip-172-31-10-98:~/terraform$ vi main.tf

```

Create [main.tf](#) file:

```
provider "aws" {  
  region = "ap-south-1"  
}
```

```
resource "aws_instance" "jenkins" {  
  ami      = "ami-0f5ee92e2d63afc18" # Ubuntu 22.04 (update  
if needed)  
  instance_type = "t2.medium"
```

```
  user_data = <<-EOF  
#!/bin/bash  
apt update -y
```

```
# Install Java  
apt install openjdk-17-jdk -y
```

```
# Install Jenkins  
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io.key | tee  
\  
/usr/share/keyrings/jenkins-keyring.asc > /dev/null
```

```
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \  
https://pkg.jenkins.io/debian-stable binary/ | tee \  
/etc/apt/sources.list.d/jenkins.list
```

```
apt update -y
```

```
apt install jenkins -y
```

```
systemctl enable jenkins
```

```
systemctl start jenkins
```

```
# Install Docker
```

```
apt install docker.io -y
```

```
systemctl start docker
```

```
usermod -aG docker jenkins
```

```
EOF
```

```
tags = {
```

```
    Name = "Jenkins-Server"
```

```
}
```

```
}
```

```

[+] Complete! Resources: 1 added, 0 changed, 0 destroyed.
ubuntu@ip-172-31-10-98:~/terraform$ cat main.tf
provider "aws" {
  region = "ap-south-1"
}

resource "aws_instance" "jenkins" {
  ami           = "ami-0f5ee92e2d63afc18" # Ubuntu 22.04 (update if needed)
  instance_type = "t3.micro"

  user_data = <<-EOF
#!/bin/bash
apt update -y

# Install Java
apt install openjdk-17-jdk -y

# Install Jenkins
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io.key | tee \
  /usr/share/keyrings/jenkins-keyring.asc > /dev/null

echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/ | tee \
  /etc/apt/sources.list.d/jenkins.list

```

Deploy : terraform init

terraform apply -auto-approve

```

ubuntu@ip-172-31-10-98:~/terraform$ vi main.tf
ubuntu@ip-172-31-10-98:~/terraform$ terraform init
terraform apply -auto-approve
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.48.0...
- Installed hashicorp/aws v6.48.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

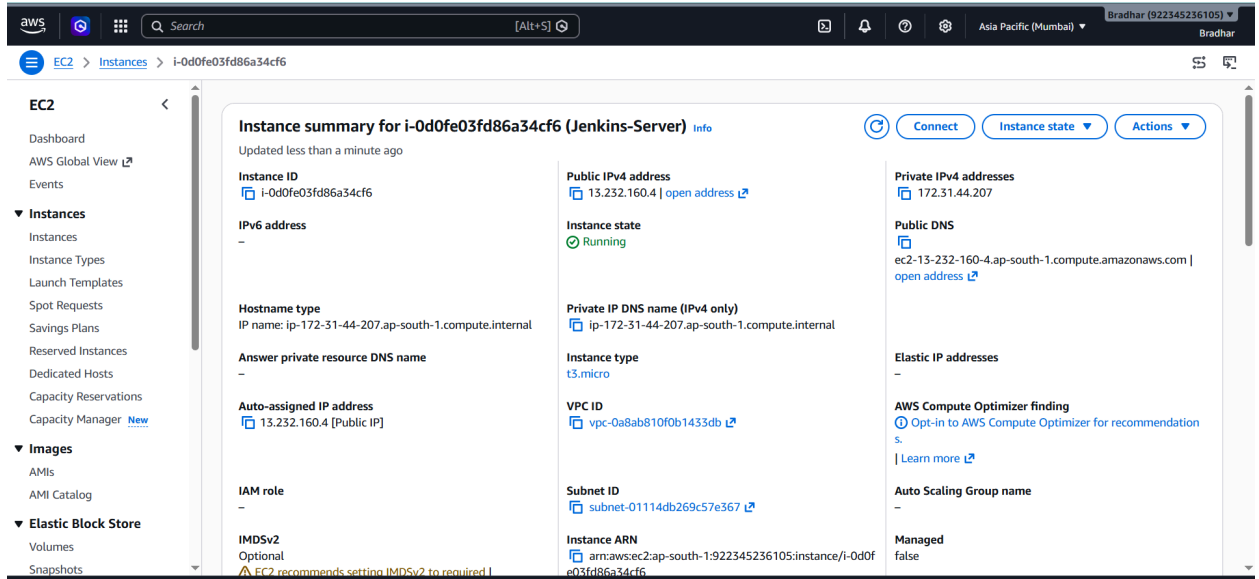
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

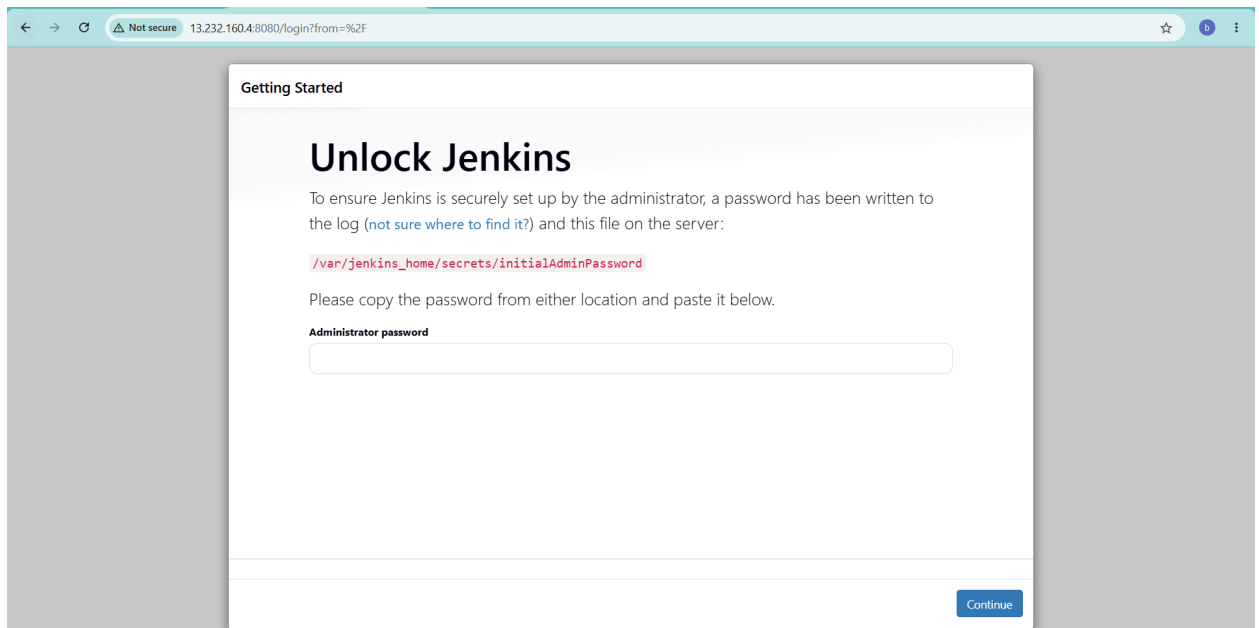
```

Verify aws console: successfully created jenkins-server



Verify : public ip:8080

We are configure the userdata in [main.tf](#) file its automatically installed the jenkins and running successfully



5. Create one Jenkins job to build and push the Docker image to Docker Hub.

- Source: <https://github.com/betawins/Python-app.git>

Clone the repo:

Cd python-app:

```
ubuntu@ip-172-31-10-98:~$ ls
Dockerfile  Python-app  custom-nginx.tar  docker-compose.yml  multi-stage-example  mumbai.pem  sample.war  terraform
ubuntu@ip-172-31-10-98:~$ cd Python-app/
ubuntu@ip-172-31-10-98:~/Python-app$ vi Dockerfile
```

Create Dockerfile:

```
ubuntu@ip-172-31-10-98:~/Python-app$ cat Dockerfile
FROM python:3.6

RUN pip install flask

COPY . /opt/

EXPOSE 8080

WORKDIR /opt

ENTRYPOINT ["python", "app.py"]
ubuntu@ip-172-31-10-98:~/Python-app$
```

```
ubuntu@ip-172-31-10-98:~/Python-app$ docker build -t node-app:v1 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/
```

```
Sending build context to Docker daemon 79.36kB
```

```
Step 1/6 : FROM python:3.6
```

```
3.6: Pulling from library/python
```

```
c4f42be2be53: Pulling fs layer
```

```
0e29546d541c: Pulling fs layer
```

```
9b829c73b52b: Pulling fs layer
```

```
cb5b7ae36172: Pulling fs layer
```

```
6494e4811622: Pulling fs layer
```

```
6f9f74896dfa: Pulling fs layer
```

```
5e3b1213efc5: Pulling fs layer
```

```
9fddfdc56334: Pulling fs layer
```

```
404f02044bac: Pulling fs layer
```

```
404f02044bac: Download complete
```

```
cb5b7ae36172: Download complete
```

```
c4f42be2be53: Download complete
```

```
9b829c73b52b: Download complete
```

```
5e3b1213efc5: Download complete
```

```
9fddfdc56334: Download complete
```

```
0e29546d541c: Download complete
```

```
6494e4811622: Download complete
```

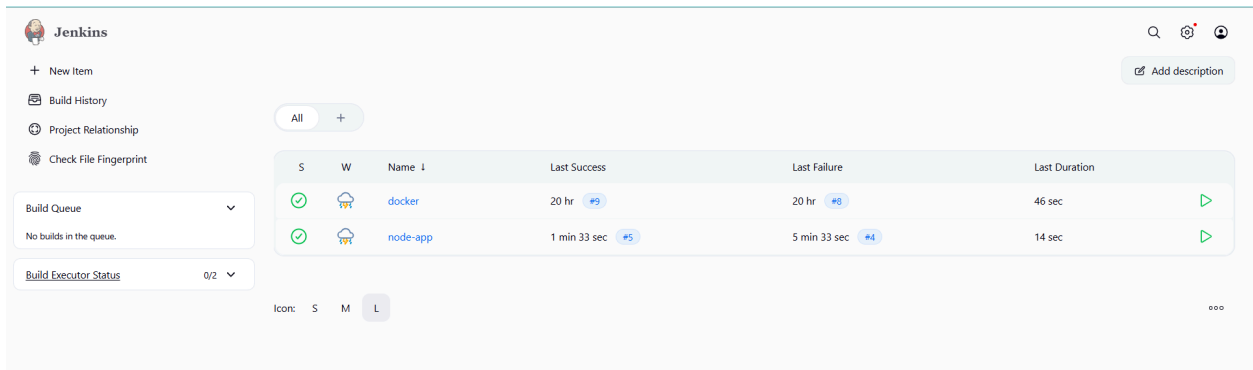
```
6f9f74896dfa: Download complete
```

```
ubuntu@ip-172-31-10-98:~/Python-app$ docker images
```

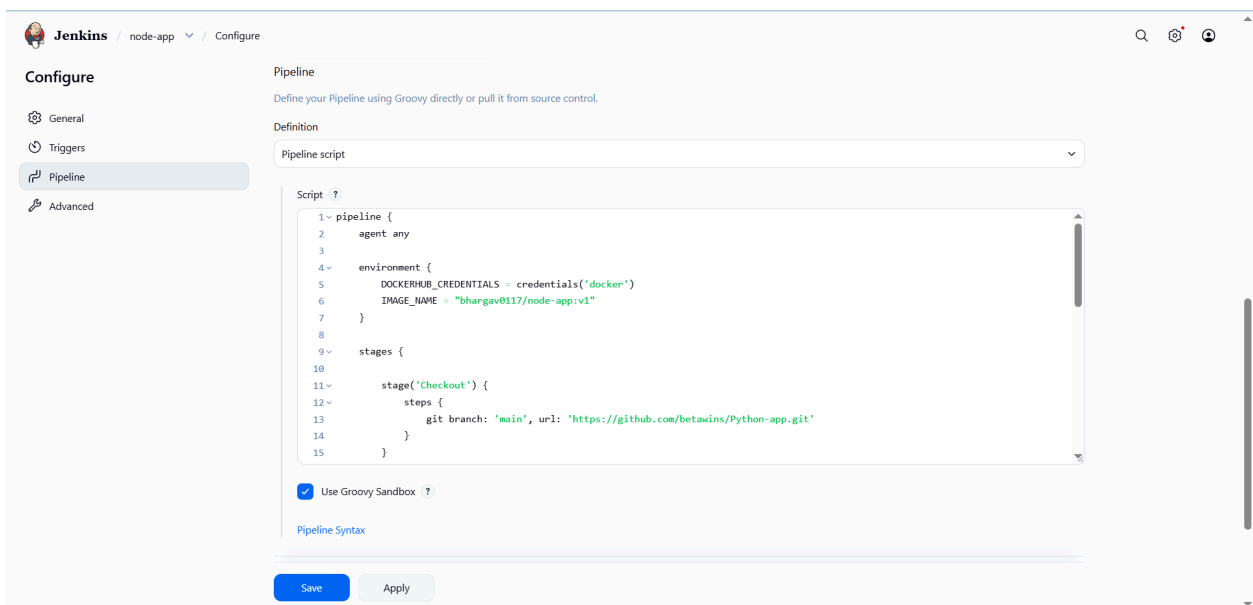
IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
922345236105.dkr.ecr.ap-south-1.amazonaws.com/bhargav0117/frontend-app:latest	ffabdf818eb	282MB	120MB	
922345236105.dkr.ecr.ap-south-1.amazonaws.com/frontend-app:latest	ffabdf818eb	282MB	120MB	
bhargav0117/custom-nginx:v1	cb423984a499	292MB	85.1MB	
bhargav0117/frontend-app:latest	675db38b5e36	282MB	120MB	
custom-nginx:v1	a84660173250	342MB	94.6MB	
eclipse-temurin:17-jre-alpine	b0ae54a36f92	257MB	69.9MB	
frontend-app:latest	ffabdf818eb	282MB	120MB	
jenkins/jenkins:lts	01c992ffef29	799MB	289MB	
maven:3.9.6-eclipse-temurin-17	29a1658b1f30	728MB	221MB	
nginx:alpine	8b1e78743a03	93.6MB	27MB	
nginx:latest	5aca99593157	241MB	66MB	
node-app:v1	a74bf23309ae	1.34GB	349MB	
node:18-alpine	8d6421d663b4	181MB	45.3MB	
python:3.6	f8652afaf88c	1.32GB	345MB	
springboot-multi:v1	a4f3cfb0c0f9	462MB	126MB	
springboot-single:v1	b578b94a53fa	871MB	283MB	
test-nginx:latest	92c0e1ba605b	238MB	63.1MB	
yourdockerhub/frontend-app:latest	675db38b5e36	282MB	120MB	

ubuntu@ip-172-31-10-98:~/Python-app\$ docker run -d -p 3000:3000 node-app:v1
b495bf78f3d0d8b2f13abbf1a3177dc9a92ce1d9a3fc4525d588ee19c1f6016
ubuntu@ip-172-31-10-98:~/Python-app\$

Create new item: node-app



Click on new item → configure → pipeline → write pipeline and save



pipeline {

agent any

environment {

DOCKERHUB_CREDENTIALS = credentials('docker')

IMAGE_NAME = "bhargav0117/node-app:v1"

```
}
```

```
stages {
```

```
    stage('Checkout') {
```

```
        steps {
```

```
            git branch: 'main', url: 'https://github.com/betawins/Python-app.git'
```

```
        }
```

```
    }
```

```
    stage('Build Docker Image') {
```

```
        steps {
```

```
            sh 'ls -la'
```

```
            sh 'docker build -t node-app:v1 .'
```

```
        }
```

```
    }
```

```
    stage('Login to Docker Hub') {
```

```
        steps {
```

```
            sh '''
```

```
                echo $DOCKERHUB_CREDENTIALS_PSW | docker login -u  
$DOCKERHUB_CREDENTIALS_USR --password-stdin
```

```
            '''
```

```
        }
```

```

    }

    stage('Push Image') {

        steps {

            sh '''

                docker tag node-app:v1 bhargav0117/node-app:v1

                docker push bhargav0117/node-app:v1

            '''

        }

    }

}

```

Build the job: build successfill

Jenkins / node-app / #5

#5 (5 Jun 2026, 06:52:06)

Started by user [admin](#)

This run spent:

- 10 ms waiting;
- 14 sec build duration;
- 14 sec total from scheduled to completion.

git Revision: a9c18a887f134751f06bab9eb128a4387b42c141
Repository: <https://github.com/betawins/Python-app.git>

- refs/remotes/origin/main

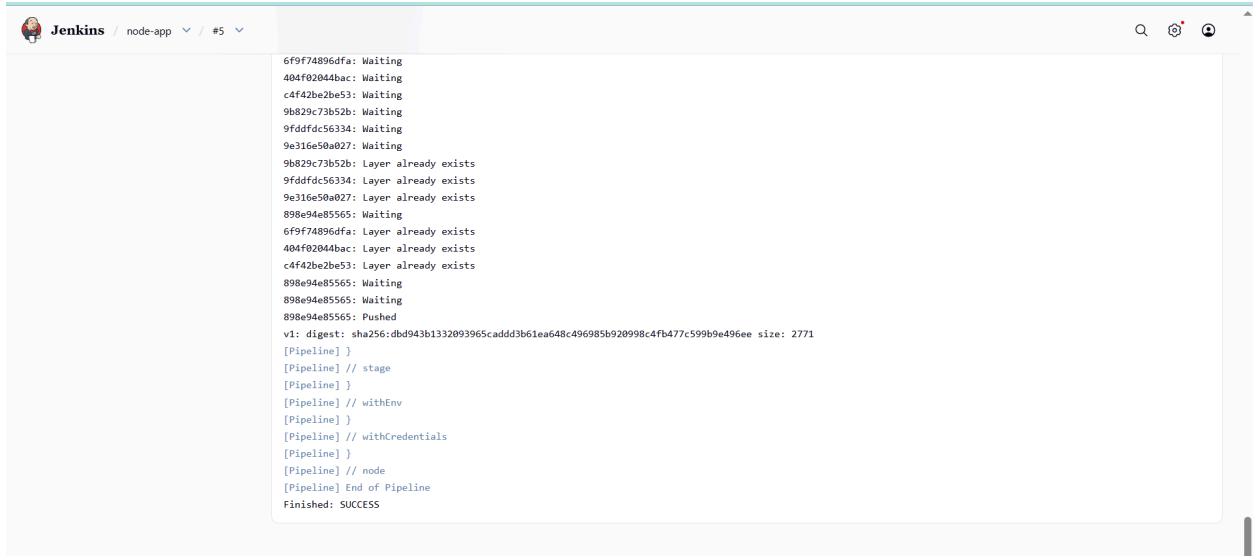
</> No changes.

Started 3 min 33 sec ago
Took [14 sec](#)

Buttons: Add description, Keep this build forever

Left sidebar: Status, Changes, Console Output, Edit Build Information, Delete build #5, Timings, Git Build Data, Pipeline Overview, Restart from Stage, Replay, Pipeline Steps, Workspaces, Previous Build

Verify console output:



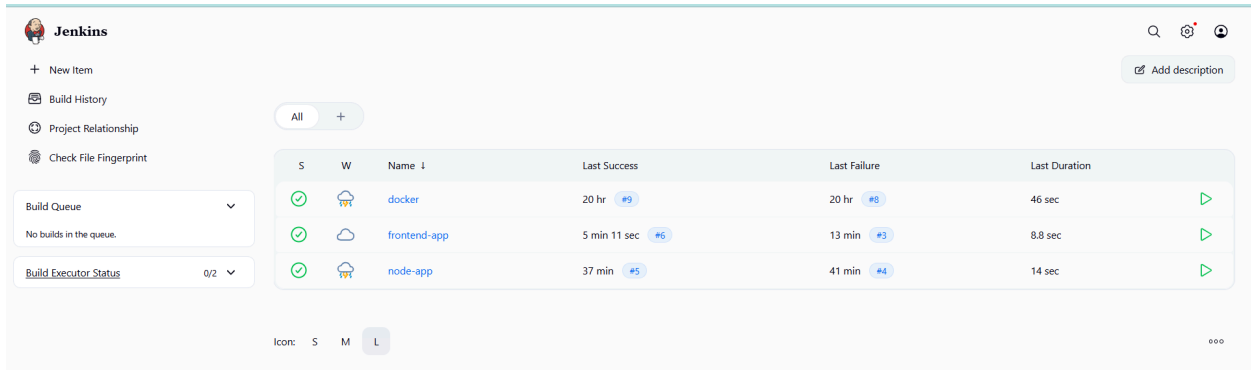
📁 **Source Codes:** <https://github.com/betawins/docker-tasks.git>

- From the **frontend source code**, write a Dockerfile, build a Docker image, run it, and push that image to your Docker registry.

Cd /docker-tasks/frontend_source

```
ubuntu@ip-172-31-10-98:~/docker-tasks/Frontend_based_source$ ls
Dockerfile  index.html  javascript.js  style.css  todayDeal.js
ubuntu@ip-172-31-10-98:~/docker-tasks/Frontend_based_source$
```

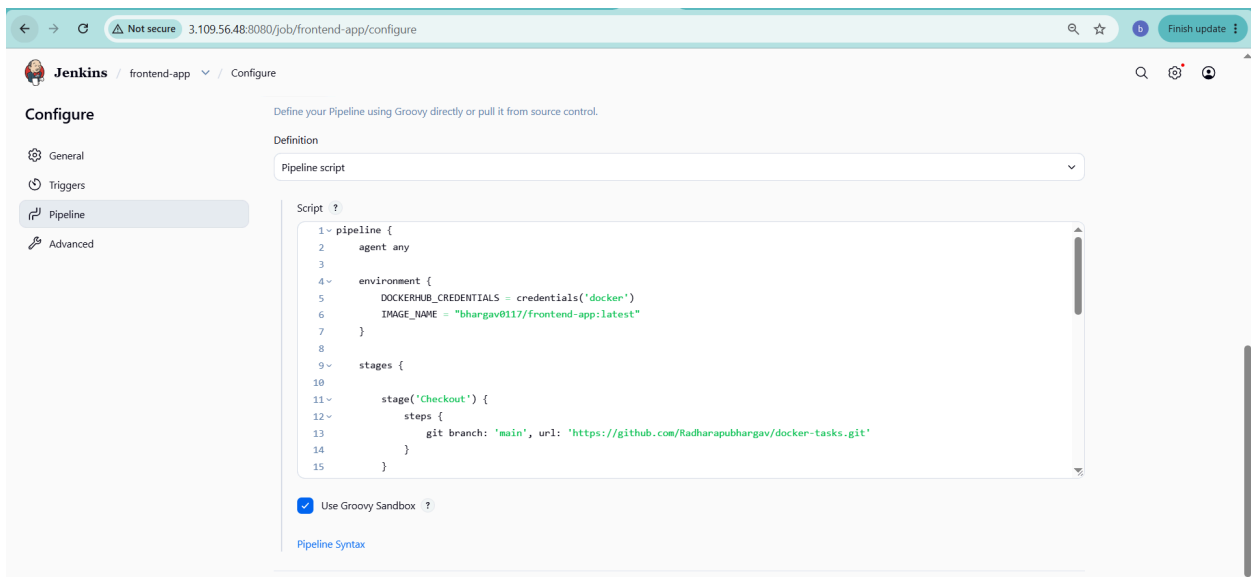
Create new item →select pipeline



The screenshot shows the Jenkins dashboard. On the left, there's a sidebar with links: 'New Item', 'Build History', 'Project Relationship', and 'Check File Fingerprint'. Below these are two dropdown menus: 'Build Queue' (showing 'No builds in the queue') and 'Build Executor Status' (showing '0/2'). The main area displays a table of build jobs. At the top right, there are search, settings, and user icons, along with an 'Add description' button. The table has columns for status (S), icon (W), name, last success, last failure, and last duration. Three jobs are listed: 'docker', 'frontend-app', and 'node-app', each with a green status icon and a play button. At the bottom, there's a filter bar with 'Icon: S M L' and a 'ooo' link.

S	W	Name	Last Success	Last Failure	Last Duration
✓		docker	20 hr #9	20 hr #8	46 sec
✓		frontend-app	5 min 11 sec #6	13 min #3	8.8 sec
✓		node-app	37 min #5	41 min #4	14 sec

Configure→write a pipeline



The screenshot shows the Jenkins 'Configure' page for a job named 'frontend-app'. The left sidebar has tabs for 'General', 'Triggers', 'Pipeline', and 'Advanced', with 'Pipeline' selected. The main area is titled 'Define your Pipeline using Groovy directly or pull it from source control.' Below this, there's a 'Definition' section with a dropdown menu set to 'Pipeline script'. A large text area contains a Groovy pipeline script. At the bottom, there's a checkbox for 'Use Groovy Sandbox' which is checked, and a link for 'Pipeline Syntax'.

```

1 pipeline {
2   agent any
3
4   environment {
5     DOCKERHUB_CREDENTIALS = credentials('docker')
6     IMAGE_NAME = "bhargav0117/frontend-app:latest"
7   }
8
9   stages {
10
11     stage('Checkout') {
12       steps {
13         git branch: 'main', url: 'https://github.com/Radharapubhargav/docker-tasks.git'
14       }
15     }
16   }
17 }
  
```

```

pipeline {
  agent any
  
```

```

environment {
  DOCKERHUB_CREDENTIALS = credentials('docker')
  IMAGE_NAME = "bhargav0117/frontend-app:latest"
}
  
```



```

stages {

    stage('Checkout') {
        steps {
            git branch: 'main', url:
'https://github.com/Radharapubhargav/docker-tasks.git'
        }
    }

    stage('Build Docker Image') {
        steps {
            sh '''
            ls -la
            ls Frontend_based_source
            docker build -t frontend-app:latest -f
Frontend_based_source/Dockerfile Frontend_based_source
            '''
        }
    }

    stage('Login to Docker Hub') {
        steps {
            sh '''
            echo $DOCKERHUB_CREDENTIALS_PSW | docker login
-u $DOCKERHUB_CREDENTIALS_USR --password-stdin
            '''
        }
    }
}

```

```
    }  
  }  
  
  stage('Push Image') {  
    steps {  
      sh '''  
        docker tag frontend-app:latest  
        bhargav0117/frontend-app:latest  
        docker push bhargav0117/frontend-app:latest  
      '''  
    }  
  }  
}  
}
```

Click on Build
Build was successfull

Jenkins / frontend-app / #6

Status #6 (5 Jun 2026, 07:24:18) Add description

Changes

Console Output

Edit Build Information

Delete build '#6'

Timings

Git Build Data

Pipeline Overview

Restart from Stage

Replay

Pipeline Steps

Workspaces

Previous Build

Started by user admin

This run spent:

- 10 ms waiting;
- 8.8 sec build duration;
- 8.8 sec total from scheduled to completion.

Revision: 4eeb2d23a859f37898936f13cf98f77ec38c4309
Repository: <https://github.com/Radharapubhargav/docker-tasks.git>

- refs/remotes/origin/main

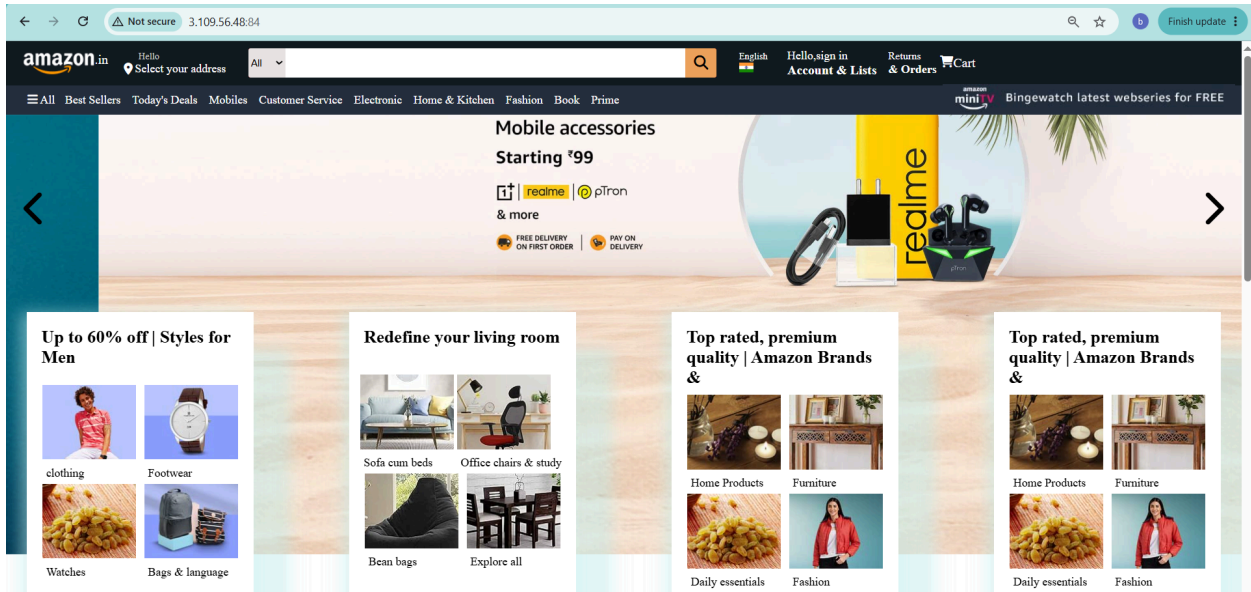
No changes.

Verify console output

Jenkins / frontend-app / #6

6a0ac1617861: Waiting
43f834d60d8a: Waiting
de1b677d8c00: Mounted from library/nginx
abaae85d1626: Waiting
a5008f4a4b25: Mounted from library/nginx
fbaed3f7fcb: Mounted from library/nginx
6a0ac1617861: Waiting
43f834d60d8a: Waiting
94d083cf706a: Mounted from library/nginx
e654dbbb9e1: Mounted from library/nginx
43f834d60d8a: Mounted from library/nginx
abaae85d1626: Mounted from library/nginx
6a0ac1617861: Waiting
6a0ac1617861: Waiting
6a0ac1617861: Mounted from bhargav0117/custom-nginx
84431a1d48da: Pushed
latest: digest: sha256:daeb76c3013573ed8662717b0cc0c9ab8804ff509a7ae1bea59ec6a18e15d0a7 size: 2262
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

Verify: Public IP:84



- From the **Java-based source code**, write a Dockerfile, build, run, and push the image to the Docker registry.

Cd /docker-tasks/java-based source code

```
ubuntu@ip-172-31-10-98:~$ cd docker-tasks/
ubuntu@ip-172-31-10-98:~/docker-tasks$ cd Java_based_source/
ubuntu@ip-172-31-10-98:~/docker-tasks/Java_based_source$ ls
pom.xml  src
ubuntu@ip-172-31-10-98:~/docker-tasks/Java_based_source$ vi Dockerfile
```

Create docker file

```
ubuntu@ip-172-31-10-98:~/docker-tasks/Java_based_source$ cat Dockerfile
# Stage 1: Build stage
FROM maven:3.9.6-eclipse-temurin-17 AS builder

WORKDIR /app

COPY . .

RUN mvn clean package -DskipTests

# Stage 2: Runtime stage
FROM eclipse-temurin:17-jdk-alpine

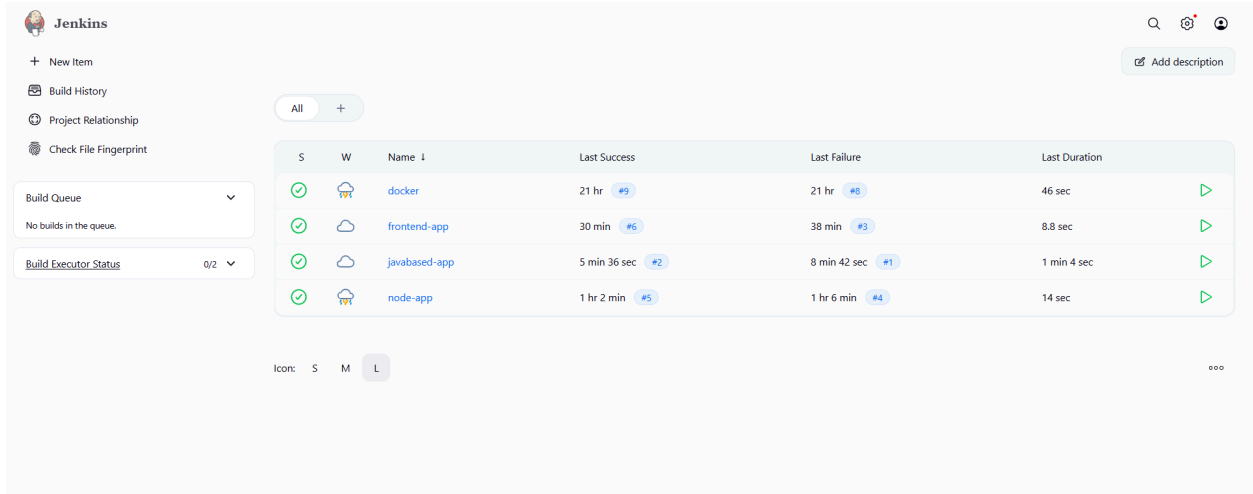
WORKDIR /app

COPY --from=builder /app/target/*.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

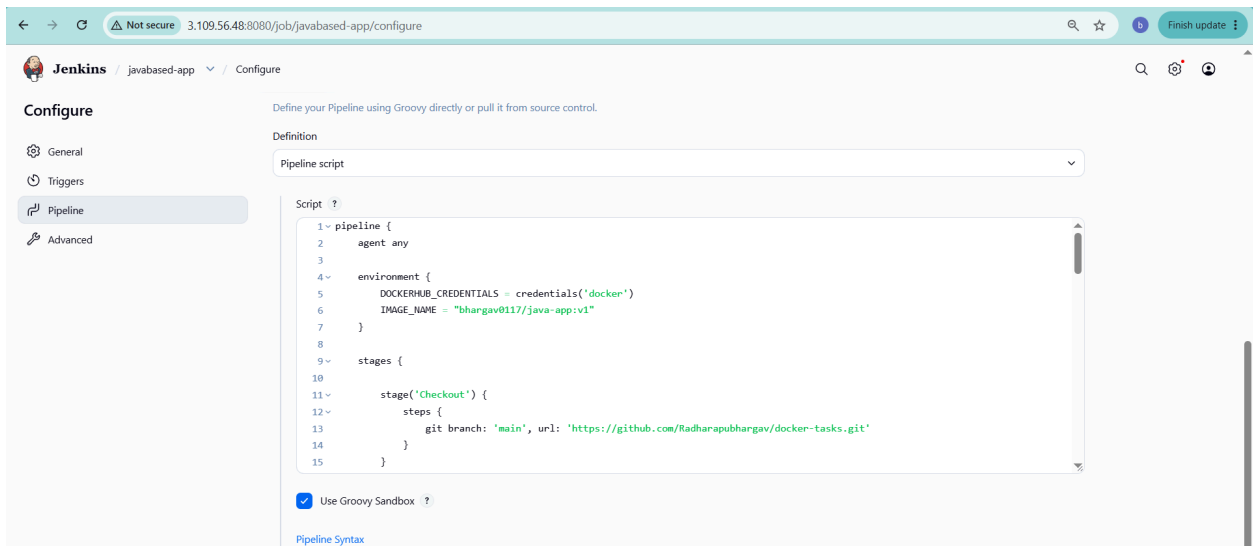
Create new item



The screenshot shows the Jenkins dashboard. On the left, there's a sidebar with links: 'New Item', 'Build History', 'Project Relationship', and 'Check File Fingerprint'. Below these are two dropdown menus: 'Build Queue' (showing 'No builds in the queue') and 'Build Executor Status' (showing '0/2'). The main area displays a table of build jobs. At the top of the table is a filter button labeled 'All'. The table has columns for status (S), icon (W), name, last success, last failure, last duration, and a play button. The jobs listed are 'docker', 'frontend-app', 'javabased-app', and 'node-app'. Below the table, there are tabs for 'Icon', 'S', 'M', and 'L', with 'L' being the active tab.

S	W	Name	Last Success	Last Failure	Last Duration
✓		docker	21 hr #9	21 hr #0	46 sec
✓		frontend-app	30 min #6	38 min #3	8.8 sec
✓		javabased-app	5 min 36 sec #2	8 min 42 sec #1	1 min 4 sec
✓		node-app	1 hr 2 min #5	1 hr 6 min #4	14 sec

Write a pipeline



The screenshot shows the Jenkins 'Configure' page for a job named 'javabased-app'. The left sidebar has tabs for 'General', 'Triggers', 'Pipeline', and 'Advanced', with 'Pipeline' being the active tab. The main area is titled 'Define your Pipeline using Groovy directly or pull it from source control.' Below this, there's a 'Definition' dropdown set to 'Pipeline script'. A text area contains the following Groovy pipeline script:

```

1 pipeline {
2   agent any
3
4   environment {
5     DOCKERHUB_CREDENTIALS = credentials('docker')
6     IMAGE_NAME = "bhargav0117/java-app:v1"
7   }
8
9   stages {
10
11     stage('Checkout') {
12       steps {
13         git branch: 'main', url: 'https://github.com/Radharapubhargav/docker-tasks.git'
14       }
15     }
16   }
17 }

```

Below the script area, there's a checkbox labeled 'Use Groovy Sandbox' which is checked. At the bottom, there's a link for 'Pipeline Syntax'.

```

pipeline {
  agent any

```

```

  environment {
    DOCKERHUB_CREDENTIALS = credentials('docker')
    IMAGE_NAME = "bhargav0117/java-app:v1"
  }

```

```
stages {

    stage('Checkout') {
        steps {
            git branch: 'main', url:
'https://github.com/Radharapubhargav/docker-tasks.git'
        }
    }

    stage('Verify Source') {
        steps {
            sh '''
            ls -la
            ls Java_based_source
            '''
        }
    }

    stage('Build Docker Image') {
        steps {
            sh '''
            cd Java_based_source
            docker build -t java-app:v1 .
            '''
        }
    }
}
```

```
}
```

```
stage('Test Run Container') {
```

```
  steps {
```

```
    sh '''
```

```
    docker run -d --name java-test -p 8082:8080 java-app:v1
```

```
    sleep 10
```

```
    docker ps
```

```
    docker stop java-test || true
```

```
    docker rm java-test || true
```

```
    '''
```

```
  }
```

```
}
```

```
stage('Login to Docker Hub') {
```

```
  steps {
```

```
    sh '''
```

```
    echo $DOCKERHUB_CREDENTIALS_PSW | docker login  
-u $DOCKERHUB_CREDENTIALS_USR --password-stdin
```

```
    '''
```

```
  }
```

```
}
```

```
stage('Tag Image') {
```

```
  steps {
```

```
    sh '''
```

```
    docker tag java-app:v1 bhargav0117/java-app:v1
    ""
  }
}

stage('Push Image') {
  steps {
    sh ""
    docker push bhargav0117/java-app:v1
    ""
  }
}

post {
  always {
    sh 'docker system prune -f || true'
  }
  success {
    echo "Java Docker Pipeline Completed Successfully 🚀"
  }
  failure {
    echo "Pipeline Failed ❌ Check logs"
  }
}
```


}

Save and build the job

Build successful

The screenshot shows the Jenkins web interface for a job named 'javabased-app'. The build is #2, dated 5 Jun 2026, 07:48:49, and is in a 'Completed' state. The left sidebar contains navigation links: Status, Changes, Console Output, Edit Build Information, Delete build #2, Timings, Git Build Data, Pipeline Overview, Restart from Stage, Replay, Pipeline Steps, Workspaces, and Previous Build. The main content area displays build details: 'Started by user admin', 'This run spent: 7 ms waiting, 1 min 4 sec build duration, 1 min 4 sec total from scheduled to completion.', and 'Repository: https://github.com/Radharapubhargav/docker-tasks.git'. A 'Create Dockerfile' button is visible. The top right shows 'Finish update' and 'Keep this build forever' options.

Verify output:

The screenshot shows the Jenkins console output for build #2. The output includes a list of deleted images with their SHA256 hashes, the total reclaimed space (65.26MB), and the final status 'Finished: SUCCESS'. The console output is as follows:

```
Deleted Images:
untagged: sha256:e668c335218ba845552e9f77b2c904c93f70225e7efae7ab30d524cfe292364
deleted: sha256:e668c335218ba845552e9f77b2c904c93f70225e7efae7ab30d524cfe292364
untagged: sha256:0d5279a3d34e9d359a17c9d2acd5a3341c6559e4c9845f7d45ef7c5c4213ee13
deleted: sha256:0d5279a3d34e9d359a17c9d2acd5a3341c6559e4c9845f7d45ef7c5c4213ee13
untagged: sha256:3cf25010cddaeb3a5effbe2cdf053ec4259b23bc2b5b5a083bc338b0cf2eafab
deleted: sha256:3cf25010cddaeb3a5effbe2cdf053ec4259b23bc2b5b5a083bc338b0cf2eafab
untagged: sha256:a2622643cdc1ec75a0b0908a67d3db1b2cab531c52d958b9f417c92a176319b7
deleted: sha256:a2622643cdc1ec75a0b0908a67d3db1b2cab531c52d958b9f417c92a176319b7
untagged: sha256:06f2b8c0e7c4318854d17666d6488916e44ebfde0b6f1c6f149e6bd02e2b0760
deleted: sha256:06f2b8c0e7c4318854d17666d6488916e44ebfde0b6f1c6f149e6bd02e2b0760
untagged: sha256:9fe70af2d0d7fda5e92b27dcf7c28c05a669b3c19eaa70f591a3e4917be35699
deleted: sha256:9fe70af2d0d7fda5e92b27dcf7c28c05a669b3c19eaa70f591a3e4917be35699

Total reclaimed space: 65.26MB
[Pipeline] echo
Java Docker Pipeline Completed Successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Verify : docker images

```
ubuntu@ip-172-31-10-98:~/docker-tasks/java_based_source$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
bhargav0117/frontend-app:latest	daeb76c30135	92.8MB	26.1MB	
bhargav0117/java-app:v1	ffef3ec4c3e7	549MB	188MB	
bhargav0117/node-app:v1	dbd943b13320	1.34GB	349MB	
custom-nginx:v1	a84660173250	342MB	94.6MB	
eclipse-temurin:17-jdk-alpine	5d14725f0e49	512MB	171MB	
eclipse-temurin:17-jre-alpine	b0ae54a36f82	257MB	68.9MB	
frontend-app:latest	daeb76c30135	92.8MB	26.1MB	
java-app:v1	ffef3ec4c3e7	549MB	188MB	
jenkins/jenkins:lts	01c992ffef29	798MB	289MB	
maven:3.9.6-eclipse-temurin-17	29a1658b1f30	728MB	221MB	
nginx:alpine	8b1e78743a03	93.6MB	27MB	
nginx:latest	5aca99593157	241MB	66MB	
node-app:v1	dbd943b13320	1.34GB	349MB	
node:18-alpine	8d6421d663b4	181MB	45.3MB	
python:3.6	f8652afaf88c	1.32GB	345MB	
springboot-multi:v1	a4f3cfc0c0f9	462MB	126MB	
springboot-single:v1	b578b94a53fa	871MB	283MB	
test-nginx:latest	92c0e1ba605b	238MB	63.1MB	

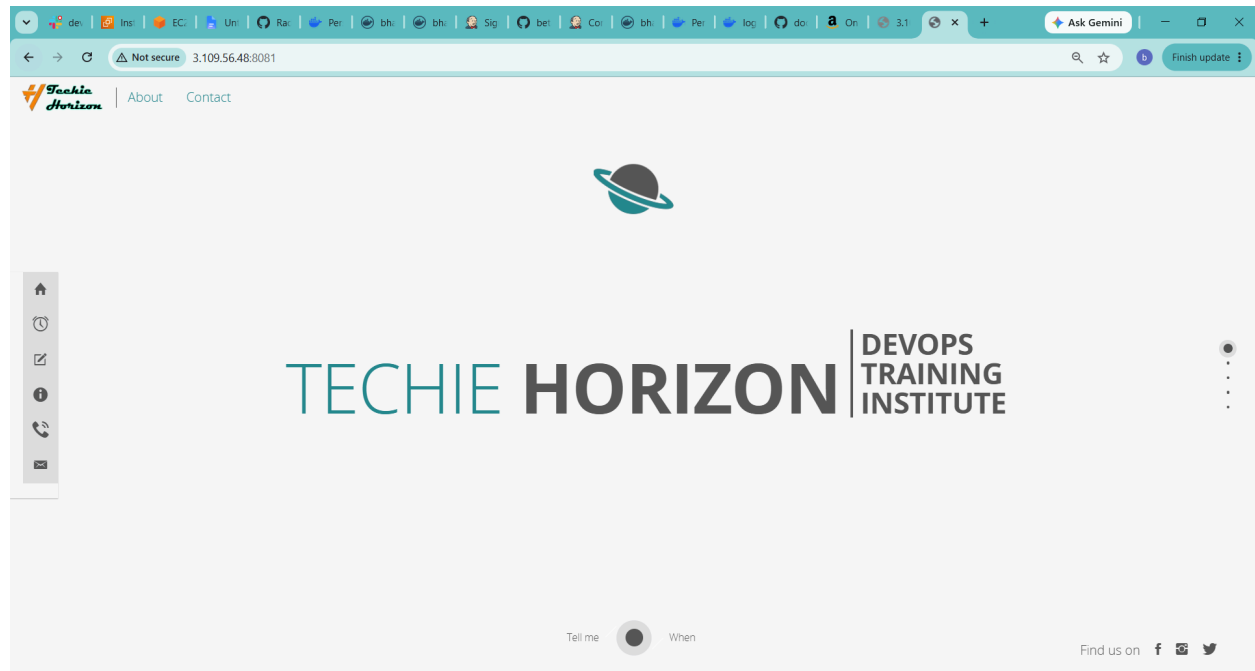
Run the container: docker run -d -p 8081:8080 java-app:v1

```
ubuntu@ip-172-31-10-98:~/docker-tasks/java_based_source$ docker run -d -p 8081:8080 java-app:v1
0e88bd5d97df5dfdd30a54a586c1b278ba58d35e2afeff763c9a1eebf9d0117
ubuntu@ip-172-31-10-98:~/docker-tasks/java_based_source$ docker ps
```

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
0e88bd5d97df	java-app:v1	gallant_yonath	"java -jar app.jar"	6 minutes ago	Up 6 minutes	0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
55c59fccb275	node-app:v1	charming_volhard	"python app.py"	20 minutes ago	Up 19 minutes	8080/tcp, 0.0.0.0:3001->3000/tcp, [::]:3001->3000/tcp
955516e708c3	frontend-app:latest	cranky_lovelace	"/docker-entrypoint..."	23 minutes ago	Up 23 minutes	0.0.0.0:84->80/tcp, [::]:84->80/tcp
83d7a323572e	8686efec0626	pensive_poincare	"/docker-entrypoint..."	54 minutes ago	Up 54 minutes	0.0.0.0:82->80/tcp, [::]:82->80/tcp
b495bf78f3d0	a74bf23309ae	jolly_rhodes	"python app.py"	2 hours ago	Up 2 hours	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp, 8080/tcp
6d49c107e2b0	675db38b8e36	laughing_chatelet	"/docker-entrypoint..."	19 hours ago	Up 19 hours	0.0.0.0:83->80/tcp, [::]:83->80/tcp
fbddd6a2be9e	jenkins/jenkins:lts	jenkins	"/usr/bin/tini -- /u..."	21 hours ago	Up 21 hours	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp

```
ubuntu@ip-172-31-10-98:~/docker-tasks/java_based_source$
```

Verify in browser: publicIP:8081



- From the **Node.js-based source code**, write a Dockerfile, build with tag `v1`, run, and push it to the Docker registry.

Cd /docker tasks/Nodejs_based_source

```
ubuntu@ip-172-31-10-98:~$ cd docker-tasks/NodeJs_based_source/
ubuntu@ip-172-31-10-98:~/docker-tasks/NodeJs_based_source$ cat Dockerfile
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm", "start"]
ubuntu@ip-172-31-10-98:~/docker-tasks/NodeJs_based_source$
```

Build: `docker build -t node-app:v1 .`

```
ubuntu@ip-172-31-10-98:~/docker-tasks/NodeJs_based_source$ docker build -t node-app:v1 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  4.17MB
Step 1/7 : FROM node:18-alpine
----> 8d6421d663b4
Step 2/7 : WORKDIR /app
----> Running in f47fd1f95da1
----> Removed intermediate container f47fd1f95da1
----> 03efcdd0e617
Step 3/7 : COPY package*.json ./
----> e44325073a10
Step 4/7 : RUN npm install
----> Running in 2760f72aa9f5
```

verify:

```
ubuntu@ip-172-31-10-98:~/docker-tasks/NodeJs_based_source$ docker images
```

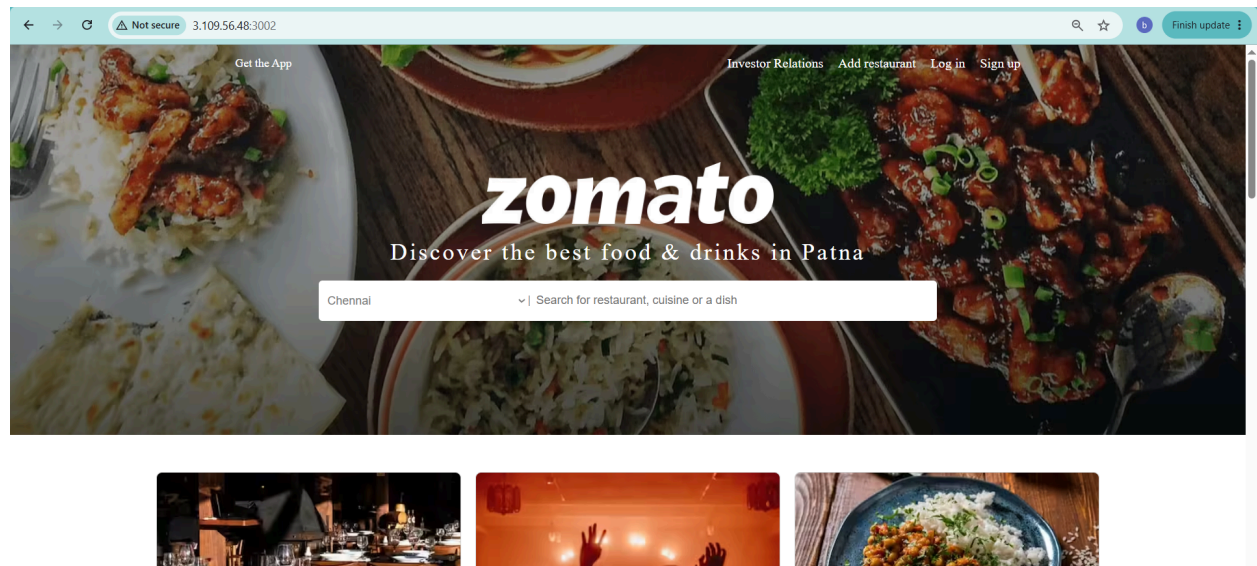
IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
bhargav0117/frontend-app:latest	daeb76c30135	92.8MB	26.1MB	U
custom-nginx:v1	a84660173250	342MB	94.6MB	
eclipse-temurin:17-jdk-alpine	5d14725f0e49	512MB	171MB	
eclipse-temurin:17-jre-alpine	b0ae54a36f82	257MB	68.9MB	
frontend-app:latest	daeb76c30135	92.8MB	26.1MB	U
java-app:v1	ffe3ec4c3e7	549MB	188MB	U
jenkins/jenkins:lts	01c992ffef29	799MB	289MB	U
maven:3.9.6-eclipse-temurin-17	29a1658b1f30	728MB	221MB	
nginx:alpine	8b1e78743a03	93.6MB	27MB	
nginx:latest	5aca99593157	241MB	66MB	
node-app:v1	ca38c837230f	993MB	169MB	
node:18-alpine	8d6421d663b4	181MB	45.3MB	
python:3.6	f8652afaf88c	1.32GB	345MB	
springboot-multi:v1	a4f3cfb0c0f9	462MB	126MB	
springboot-single:v1	b578b94a53fa	871MB	283MB	
test-nginx:latest	92c0e1ba605b	238MB	63.1MB	

Run: `docker run -d -p 3001:3000 node-app:v1`

```
ubuntu@ip-172-31-10-98:~/docker-tasks/NodeJs_based_source$ docker run -d -p 3001:3000 node-app:v1
4798c009341af283c0be35922c51ffaa54ae94d531420df637ab9c6dbae702ba
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint frosty_archimedes (c1afd7aeb42830d7c34b274566392a6f70ac5750d0b6fae5d0e623a4c1bcbf3): Bind for 0.0.0.0:3001 failed: port is already allocated

Run 'docker run --help' for more information
ubuntu@ip-172-31-10-98:~/docker-tasks/NodeJs_based_source$ docker run -d -p 3002:3000 node-app:v1
b0730a7ade36d6f4b6bd7f8e2f55d30b2861d40a5c24cf904109bee9650c5a1c
ubuntu@ip-172-31-10-98:~/docker-tasks/NodeJs_based_source$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
b0730a7ade36   node-app:v1    "docker-entrypoint.s..." 8 seconds ago  Up 7 seconds  0.0.0.0:3002->3000/tcp, [::]:3002->3000/tcp
0e88bd5d97df   java-app:v1    "java -jar app.jar"        2 hours ago   Up 2 hours    0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
55c59f6cb275   dbd943013320  "python app.py"           2 hours ago   Up 2 hours    8080/tcp, 0.0.0.0:3001->3000/tcp, [::]:3001->3000/tcp
```

Verify in browser: Public IP:3001



- Write a **docker-compose** file to set up WordPress with a MySQL database.

Check docker compose version

```
ubuntu@ip-172-31-10-98:~$ docker compose version
Docker Compose version 2.40.3+ds1-0ubuntu1
```

Create docker-compose.yml file

version: "3.8"

services:

db:

image: mysql:8.0

container_name: wordpress-db

restart: always

environment:

MYSQL_ROOT_PASSWORD: rootpass

MYSQL_DATABASE: wordpress

MYSQL_USER: wpuser

MYSQL_PASSWORD: wppass

volumes:

- db_data:/var/lib/mysql

networks:

- wp-network

wordpress:

image: wordpress:latest

container_name: wordpress-app

restart: always

depends_on:

- db

ports:

- "8080:80"

environment:

WORDPRESS_DB_HOST: db:3306

WORDPRESS_DB_USER: wpuser

WORDPRESS_DB_PASSWORD: wppass

WORDPRESS_DB_NAME: wordpress

volumes:

- wp_data:/var/www/html

networks:

- wp-network

volumes:

db_data:

wp_data:

networks:

wp-network:

```
ubuntu@ip-172-31-10-98:~$ cat docker-compose.yml
version: "3.8"
```

```
services:
```

```
  db:
    image: mysql:8.0
    container_name: wordpress-db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql
    networks:
      - wp-network
```

```
  wordpress:
    image: wordpress:latest
    container_name: wordpress-app
    restart: always
    depends_on:
      - db
    ports:
      - "8084:80"
```

Run : docker compose up -d

```
ubuntu@ip-172-31-10-98:~$ docker compose up -d
WARN[0000] /home/ubuntu/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 40/40
 ✓ wordpress Pulled                                19.9s
 ✓ db Pulled                                         17.2s
```

Verify in browser: public IP:8084

